Reporte Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente–servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guia practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación

- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos)

Este módulo se desarrolla de forma independiente

Actividad

1. Crear un archivo:
   servidor.py
2. Crear un archivo auxiliar:
   entrada_servidor.txt
3. Implementar un programa que:
   - Revise periódicamente el archivo entrada_servidor.txt.
   - Detecte si existe contenido nuevo.
   - Procese el mensaje:
     - Ejemplo: convertir texto a mayúsculas o invertirlo.
   - Escriba el resultado

   en: salida_servidor.txt
4. Condiciones:
   - Debe ejecutarse continuamente (simulación de servicio activo).
   - No debe reprocesar el mismo mensaje.
   - Debe manejar archivos vacíos.

◆ **Parte 2: Desarrollo del CLIENTE (1 hora)**

Se desarrolla sin necesidad de ver el código del servidor

Actividad

1. Crear archivo:
   cliente.py
2. Implementar un programa que:
   - Solicite un mensaje al usuario.
   - Escriba el mensaje

   en: entrada_servidor.txt
   - Espere un tiempo (simulando latencia).
   - Lea la respuesta

   desde: salida_servidor.txt
   - Muestre el resultado en pantalla.
3. Condiciones:
   - No debe asumir cómo funciona el servidor internamente.
   - Solo debe conocer los archivos de entrada y salida.

◆ **Parte 3: Integración del sistema (25 minutos)**

1. Ejecutar primero el servidor.

2. Ejecutar el cliente.
3. Enviar varios mensajes.
4. Verificar:
    - Flujo correcto
    - Respuesta adecuada

NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

6. Resultados esperados:

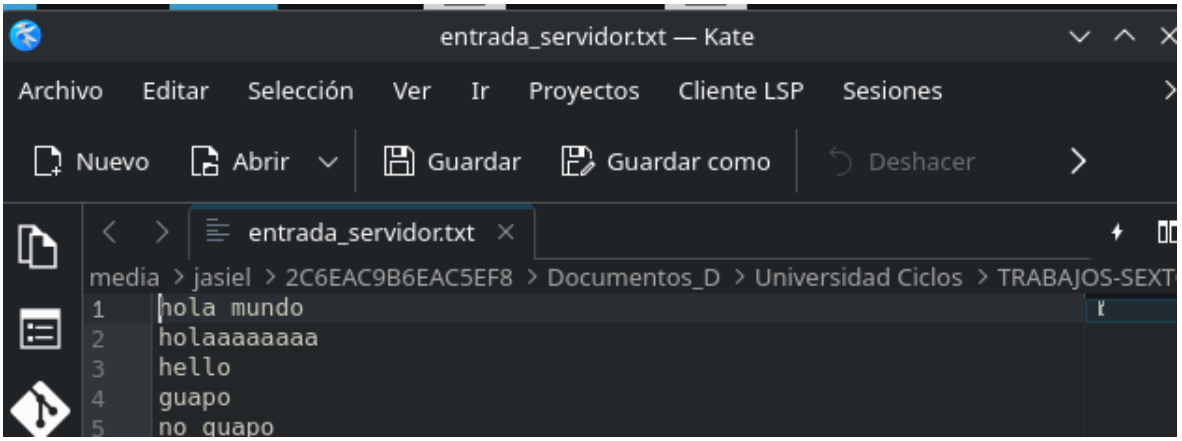
Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git).
- Capturas de pantalla que evidencien: (a) el archivo entrada_servidor.txt siendo llenado por el cliente, (b) salida_servidor.txt con el resultado procesado, (c) al menos 3 mensajes procesados correctamente.
- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).

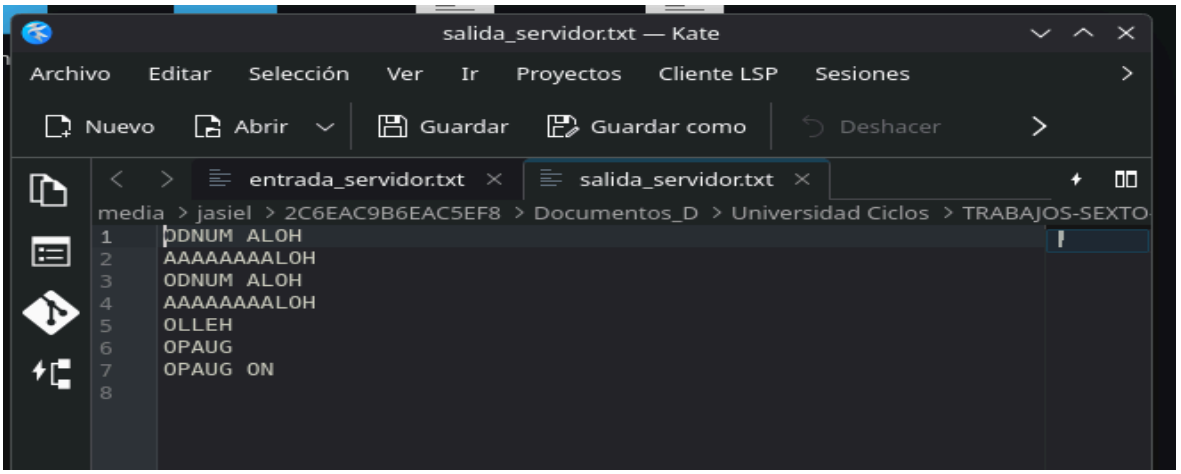
Codigo Fuente: https://github.com/Jasiel2023/Portafolio_SistemasDistribuidos/tree/main/UNIDAD%201/APE_2

Capturas de Pantalla:

- Archivo entrada_servidor.txt, con datos que fueron llenados



- Archivo salida_servidor, con el resultado procesado



7. **Preguntas de Control:**

¿Qué significa que cliente y servidor estén desacoplados?

Significa que tanto el cliente como el servidor son independientes entre sí, si el servidor se llega a caer, el cliente sigue estando en marcha.

¿Qué ventajas tiene desarrollar componentes por separado?

Desarrollar los componentes por separado permite organizar mejor el código, facilitar su mantenimiento y detectar errores con mayor facilidad. Además, cada componente puede modificarse o actualizarse sin afectar directamente al otro, siempre que se mantenga la misma forma de comunicación. También permite reutilizar código y distribuir las tareas de desarrollo, haciendo que el trabajo sea más ordenado y eficiente.

¿Qué problemas pueden surgir al integrar sistemas independientes?

Al integrar sistemas independientes pueden surgir problemas de comunicación, incompatibilidad en los formatos de los mensajes, errores en las rutas de los archivos y dificultades de sincronización. Además, es necesario asegurarse de que los componentes estén correctamente configurados y que los archivos compartidos se encuentren en la ubicación indicada. Si uno de los componentes no se ejecuta correctamente, la comunicación entre el cliente y el servidor puede verse afectada

¿Qué sucede si el servidor no está ejecutándose?

Si el servidor no está ejecutándose, el cliente puede escribir el mensaje en el archivo de entrada, pero no se realizará el procesamiento correspondiente ni se generará una respuesta nueva. En consecuencia, el cliente podría quedarse esperando o leer una respuesta anterior si el programa no verifica correctamente la actualización del archivo de salida. Por esta razón, es importante comprobar que el servidor esté activo antes de iniciar la comunicación.

¿Cómo se simula la comunicación en esta práctica?

La comunicación se simula mediante archivos de texto compartidos entre el cliente y el servidor. El cliente escribe el mensaje en el archivo entrada_servidor.txt, mientras que el servidor revisa periódicamente este archivo, procesa el contenido y guarda el resultado en salida_servidor.txt. Finalmente, el cliente lee la respuesta y la muestra en pantalla. De esta manera, se representa el intercambio de información entre dos componentes sin necesidad de utilizar una conexión de red directa.

¿Qué limitaciones tiene este modelo frente a uno real?

Este modelo tiene limitaciones porque la comunicación depende de archivos compartidos y no ocurre necesariamente en tiempo real. El servidor debe revisar periódicamente los archivos, lo que puede generar retrasos en la respuesta. También pueden presentarse conflictos si varios clientes escriben al mismo tiempo, así como problemas si los archivos se eliminan, se bloquean o dejan de estar disponibles. A diferencia de los sistemas distribuidos reales que utilizan protocolos de red, este modelo es una simulación sencilla y no ofrece mecanismos avanzados de seguridad, sincronización y gestión de múltiples conexiones.

¿Cómo se podría mejorar la sincronización?

La sincronización podría mejorarse mediante el uso de identificadores únicos para cada mensaje, con el fin de relacionar correctamente cada solicitud con su respuesta. También se podrían implementar mecanismos de bloqueo de archivos para evitar que varios clientes escriban al mismo

tiempo, además de controlar los tiempos de espera y verificar si el servidor ya procesó el mensaje. Otra mejora sería utilizar una cola de mensajes o un protocolo de comunicación como TCP, que permitiría gestionar los intercambios de información de una manera más organizada y confiable.

¿Qué pasaría si múltiples clientes usan el sistema?

Si varios clientes utilizan el sistema al mismo tiempo, podrían producirse conflictos al escribir en el archivo entrada_servidor.txt o al leer las respuestas de salida_servidor.txt. Un mensaje podría sobrescribir a otro y un cliente podría recibir una respuesta que no le corresponde. Para evitar estos problemas, sería necesario implementar identificadores para las solicitudes, mecanismos de sincronización y una forma de gestionar los mensajes de manera ordenada, garantizando que cada cliente reciba la respuesta correspondiente a su petición.

8. Bibliografía

Semana 2. Guaman, Jose. 2026